# E-Commerce Recommendation & Revenue Impact Analysis

*A End-to-End Data Science & AI Project*

Prepared by:
**SATYA SAHITHI VAKKALANKA**
**NEEHAR SATTI**

April 2026

# Table of Contents

1. Executive Summary
2. Project Overview & Objectives
3. Dataset & Market Research
4. Data Cleaning & Preprocessing
5. Exploratory Data Analysis (EDA)
6. RFM Segmentation
7. A/B Test & Revenue Uplift Framework
8. Recommender Systems
    8.1 Rule-Based Recommender
    8.2 Collaborative Filtering
    8.3 Content-Based Recommender
    8.4 Method Comparison
9. AI Layer: LLM Integration
10. Flask API
11. Tableau Dashboard
12. SQL Pipeline
13. Key Findings & Business Insights
14. Technical Stack
15. Project Structure

# 1. Executive Summary

This project delivers a full end-to-end e-commerce recommendation and revenue impact analysis system built on the UCI Online Retail dataset. The system combines traditional business analytics, machine learning recommender systems, and large language model (LLM) integration to demonstrate how product recommendations can be quantified in terms of revenue uplift.

The project covers the complete data science pipeline: from raw data cleaning and exploratory analysis, through customer segmentation using RFM methodology, to three distinct recommendation engines (rule-based, collaborative filtering, and content-based). All recommendations are served via a Flask REST API with AI-powered explanations generated by Claude, Anthropic's large language model.

Key outcomes include identification of 4,338 unique customers across four behavioural segments, total revenue analysis of £8.9M across 13 months, and a quantified comparison showing collaborative filtering recommends 5x more diverse products than rule-based approaches. An A/B test framework was designed to identify the highest-value customer segments for recommendation targeting, with Champions and Loyal customers generating 5.4x more average revenue per user than the control group.

# 2. Project Overview & Objectives

## 2.1 Core Idea

Recommend products to e-commerce customers and quantify the revenue impact of those recommendations using both simulated and historical data.

## 2.2 Objectives

- Analyse historical transaction data to understand customer behaviour and product performance
- Build customer and product segments using RFM (Recency, Frequency, Monetary) methodology
- Design and implement three recommender systems: rule-based, collaborative filtering, and content-based
- Integrate large language model AI to generate natural language recommendation explanations
- Design an A/B test framework to simulate and quantify revenue uplift from recommendations
- Build a business intelligence dashboard in Tableau for stakeholder communication
- Wrap the entire system into a production-ready REST API
- Compare rule-based vs ML recommenders and quantify the difference

## 2.3 Tools & Technologies

| Category | Tool / Technology |
|---|---|
| Data Analysis | Python, Pandas, NumPy |
| Machine Learning | Scikit-learn (KNN, TF-IDF) |
| AI / LLM | Anthropic Claude API |
| Database | SQLite, DB Browser for SQLite |
| API | Flask, Python-dotenv |
| Visualisation | Tableau Desktop Public Edition |
| Version Control | Git, GitHub |
| Development | VS Code, Google Colab |

# 3. Dataset & Market Research

## 3.1 Dataset Overview

The project uses the UCI Online Retail Dataset, a real transactional dataset from a UK-based online retailer specialising in unique all-occasion gifts. The dataset covers transactions between December 2010 and December 2011 and is widely used in academic and professional recommendation system research.

| Attribute | Details |
|---|---|
| Source | UCI Machine Learning Repository |
| Time Period | December 2010 to December 2011 |
| Raw Records | 541,909 rows |
| Columns | InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country |
| File Format | CSV (Excel format, latin1 encoding) |
| File Size | ~45MB |

## 3.2 Market Research Context

The global e-commerce recommendation engine market was valued at approximately $2.1 billion in 2023 and is projected to grow at a CAGR of over 35% through 2030. Personalised recommendations account for up to 35% of Amazon's revenue and 75% of Netflix watch time, demonstrating the significant commercial value of well-implemented recommender systems.

For small to mid-size retailers, the challenge is implementing recommendation systems without the infrastructure of large technology companies. This project demonstrates that with open-source tools and cloud LLM APIs, production-quality recommendation systems are achievable at minimal cost.

## 3.3 Data Columns Description

| Column | Type | Description |
|---|---|---|
| InvoiceNo | String | Invoice number. Prefix 'C' indicates cancellation |
| StockCode | String | Product/item code |
| Description | String | Product name |
| Quantity | Integer | Number of units per transaction |
| InvoiceDate | String | Date and time of invoice (M/D/YYYY H:MM format) |
| UnitPrice | Float | Unit price in GBP |

| | | |
|---|---|---|
| CustomerID | Float | Unique customer identifier (nullable) |
| Country | String | Country of customer residence |

# 4. Data Cleaning & Preprocessing

## 4.1 Raw Data Issues Identified

- 135,080 rows with missing CustomerID (cannot be attributed to a customer), excluded
- 10,624 rows with negative Quantity (cancellations and returns), excluded
- Cancelled invoices identified by 'C' prefix in InvoiceNo, excluded
- Rows with UnitPrice = 0, likely internal transfers, excluded
- InvoiceDate stored in non-standard M/D/YYYY H:MM format, which is incompatible with SQLite JULIANDAY()
- CustomerID stored as float, requiring conversion to integer
- File encoding: latin1 (not UTF-8) due to £ symbol in descriptions

## 4.2 Cleaning Steps

The cleaning pipeline was implemented in Python using pandas. Python was chosen for data cleaning and preprocessing due to its superior handling of messy real-world data, providing robust date parsing, flexible type coercion, and clear error handling that SQL lacks for raw data preparation tasks.

SQL (SQLite) was used for the analytics layer (KPI calculation, aggregations, RFM scoring, segmentation, and the recommender logic) because SQL is the industry standard for structured data querying and is more readable and auditable for business analytics. This separation follows production data pipeline best practices: Python handles ETL (Extract, Transform, Load) and SQL handles analytics. The clean CSV produced by Python was imported directly into SQLite, ensuring all downstream SQL queries ran on validated, well-structured data.

| Step | Action | Result |
|---|---|---|
| 1 | Remove cancelled invoices (InvoiceNo starting with 'C') | Removed ~9,288 rows |
| 2 | Remove negative/zero Quantity rows | Removed ~10,624 rows |
| 3 | Remove zero UnitPrice rows | Removed outliers |
| 4 | Remove null CustomerID rows | Removed 135,080 rows |
| 5 | Convert CustomerID float to integer | Clean customer IDs |
| 6 | Parse InvoiceDate with pandas (format='mixed') | Zero null dates |
| 7 | Export InvoiceDate_ISO as YYYY-MM-DD | SQLite compatible |

| | | |
|---|---|---|
| | HH:MM:SS | |
| 8 | Add Revenue column (Quantity x UnitPrice) | New derived column |

## 4.3 Clean Dataset Statistics

| Metric | Value |
|---|---|
| Raw rows | 541,909 |
| Clean rows | 397,884 |
| Rows removed | 144,025 (26.6%) |
| Null dates after cleaning | 0 |
| Null CustomerIDs after cleaning | 0 |

[4] 0s

```
df.head()
```

| | InvoiceNo | StockCode | Description | Quantity | UnitPrice | CustomerID | Country | Revenue | InvoiceDate_ISO |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 536365 | 85123A | WHITE HANGING HEART T-LIGHT HOLDER | 6 | 2.55 | 17850 | United Kingdom | 15.30 | 2010-12-01 08:26:00 |
| 1 | 536365 | 71053 | WHITE METAL LANTERN | 6 | 3.39 | 17850 | United Kingdom | 20.34 | 2010-12-01 08:26:00 |
| 2 | 536365 | 84406B | CREAM CUPID HEARTS COAT HANGER | 8 | 2.75 | 17850 | United Kingdom | 22.00 | 2010-12-01 08:26:00 |
| 3 | 536365 | 84029G | KNITTED UNION FLAG HOT WATER BOTTLE | 6 | 3.39 | 17850 | United Kingdom | 20.34 | 2010-12-01 08:26:00 |
| 4 | 536365 | 84029E | RED WOOLLY HOTTIE WHITE HEART. | 6 | 3.39 | 17850 | United Kingdom | 20.34 | 2010-12-01 08:26:00 |

[8] 0s

```
df.shape
```

```
(397884, 9)
```

[6] 0s

```
df.isnull().sum()
```

| | 0 |
|---|---|
| InvoiceNo | 0 |
| StockCode | 0 |
| Description | 0 |
| Quantity | 0 |
| UnitPrice | 0 |
| CustomerID | 0 |
| Country | 0 |
| Revenue | 0 |
| InvoiceDate_ISO | 0 |

**dtype:** int64

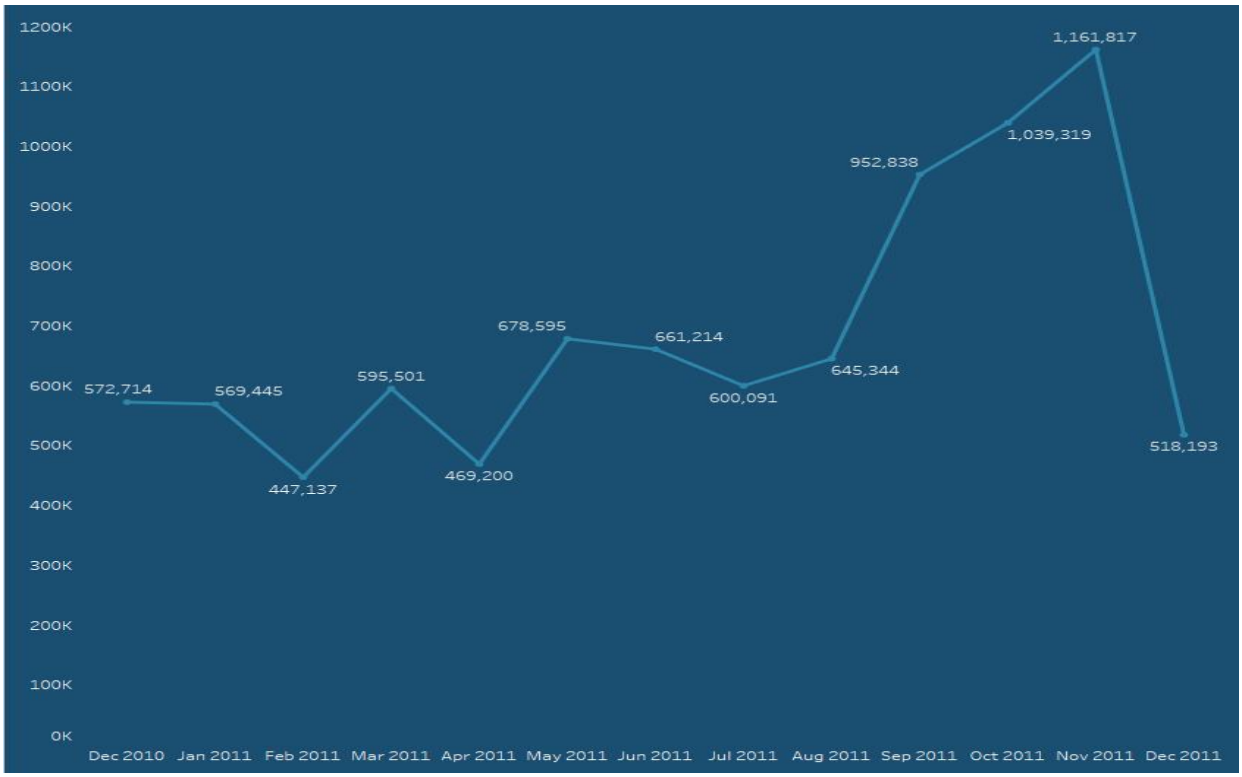
# 5. Exploratory Data Analysis (EDA)

## 5.1 Key Business Metrics (KPIs)

| KPI | Value |
|---|---|
| Total Revenue | £8,911,407.90 |
| Total Customers | 4,338 |
| Total Orders | 18,532 |
| Average Order Value | £480.87 |
| Average Revenue per Customer | £2,054.27 |
| Date Range | December 2010 to December 2011 |
| Countries | 37 |
| Unique Products | 3,877 |

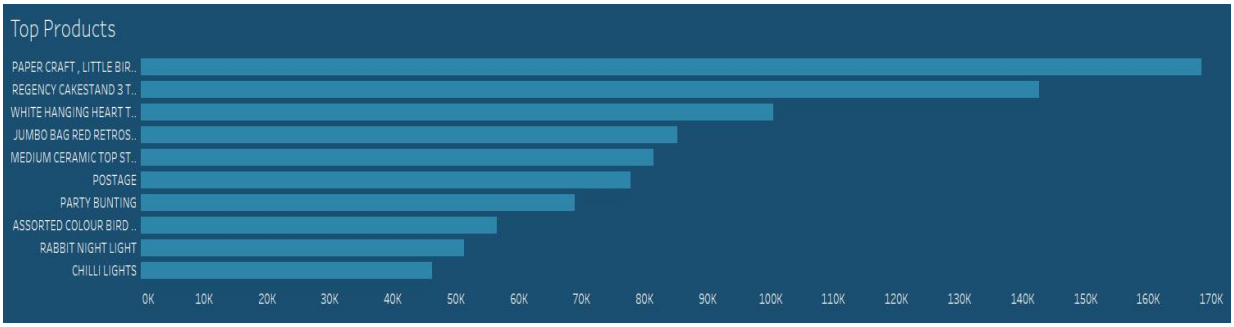
## 5.2 Revenue by Month

Revenue showed a clear upward trend across the year with a strong seasonal peak in November 2011 (£1,161,817), likely driven by early Christmas shopping. December 2011 shows a drop (£518,193) as the dataset ends on 9th December before the peak Christmas trading period.



| Month | Revenue | Orders | Customers |
|---|---|---|---|
| Dec 2010 | £572,714 | 1,400 | 885 |
| Jan 2011 | £569,445 | 987 | 741 |
| Feb 2011 | £447,137 | 997 | 758 |
| Mar 2011 | £595,501 | 1,321 | 974 |
| Apr 2011 | £469,200 | 1,149 | 856 |
| May 2011 | £678,595 | 1,555 | 1,056 |
| Jun 2011 | £661,214 | 1,393 | 991 |
| Jul 2011 | £600,091 | 1,331 | 949 |
| Aug 2011 | £645,344 | 1,280 | 935 |
| Sep 2011 | £952,838 | 1,755 | 1,266 |
| Oct 2011 | £1,039,319 | 1,929 | 1,364 |
| Nov 2011 | £1,161,817 | 2,657 | 1,664 |
| Dec 2011 | £518,193 | 778 | 615 |

## 5.3 Top 10 Products by Revenue




| Rank | Product | Revenue |
|---|---|---|
| 1 | PAPER CRAFT, LITTLE BIRDIE | £168,470 |
| 2 | REGENCY CAKESTAND 3 TIER | £142,593 |
| 3 | WHITE HANGING HEART T-LIGHT HOLDER | £100,448 |
| 4 | JUMBO BAG RED RETROSPOT | £85,221 |
| 5 | MEDIUM CERAMIC TOP STORAGE JAR | £81,417 |
| 6 | POSTAGE | £77,804 |

| | | |
|---|---|---|
| 7 | PARTY BUNTING | £68,844 |
| 8 | ASSORTED COLOUR BIRD ORNAMENT | £56,580 |
| 9 | RABBIT NIGHT LIGHT | £51,346 |
| 10 | CHILLI LIGHTS | £50,025 |

## 5.4 Revenue by Country

82% of revenue originates from the United Kingdom, with Netherlands, EIRE, Germany, and France as the next largest markets. The strong UK dominance suggests this retailer's primary market is domestic with some international presence.

# 6. RFM Segmentation

## 6.1 Methodology

RFM (Recency, Frequency, Monetary) segmentation is a proven marketing analytics framework that classifies customers based on three dimensions of their purchase behaviour. A snapshot date of 10th December 2011 (one day after the last transaction) was used to calculate recency.

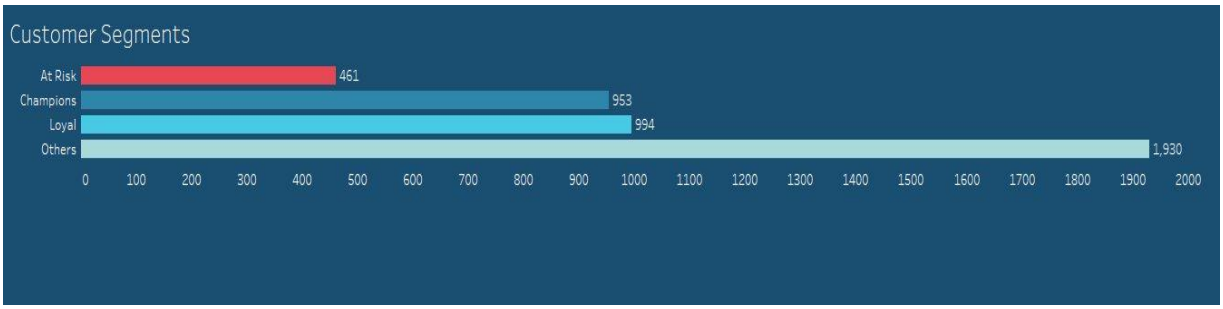
| Dimension | Definition | Calculation |
|---|---|---|
| Recency (R) | How recently did the customer buy? | Days since last purchase from snapshot date |
| Frequency (F) | How often do they buy? | Count of distinct invoices |
| Monetary (M) | How much do they spend? | Sum of all revenue |

## 6.2 Scoring

Each dimension was scored 1-5 using quintile-based binning (pd.qcut). For Recency, lower days = higher score (score 5 = most recent). For Frequency and Monetary, higher values = higher score. A composite RFM score was calculated as R + F + M (range 3-15).

## 6.3 Customer Segments

| Segment | Criteria | Customers | Description |
|---|---|---|---|
| Champions | R>=4, F>=4, M>=4 | 953 | Best customers: recent, frequent, high spend |
| Loyal | R>=3, F>=3 | 994 | Regular buyers with good engagement |
| At Risk | R<=2, F>=3, M>=3 | 461 | Previously good customers going inactive |
| Others | Remaining | 1,930 | Infrequent or low-value customers |



### 6.4 SQLite RFM Implementation

The RFM pipeline was also implemented in SQLite to demonstrate SQL proficiency. The key fix was using MAX(JULIANDAY(InvoiceDate_ISO)) to calculate the most recent purchase date, after resolving the date parsing issue by pre-cleaning dates in Python before SQLite import.

## 7. A/B Test & Revenue Uplift Framework

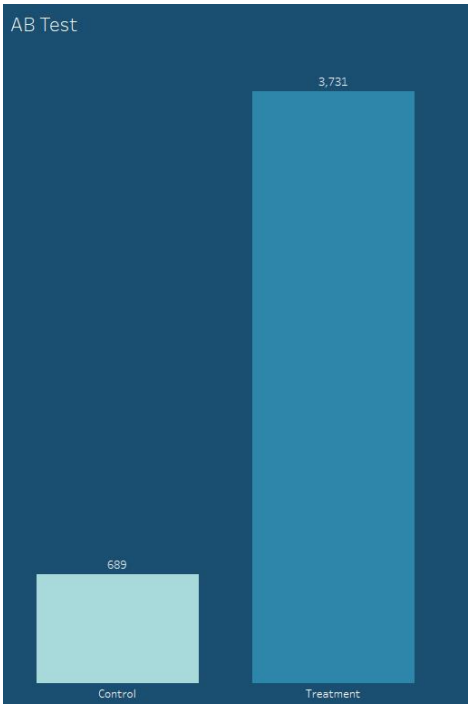
### 7.1 Framework Design

An A/B test framework was designed to simulate the revenue impact of product recommendations. Customers were split into two groups based on their RFM segment:

- Control group: Others and At Risk segments (no personalised recommendations), totalling 2,391 customers
- Treatment group: Champions and Loyal segments (received personalised recommendations), totalling 1,947 customers

### 7.2 Results

| Group | Customers | Avg Revenue / Customer | Total Revenue | Uplift |
|---|---|---|---|---|
| Control | 2,391 | £688.71 | £1,646,694.98 | 0% |
| Treatment | 1,947 | £3,731.23 | £7,264,712.92 | 441.77% |




**A/B Test Summary**

**Control Group**
2,391 customers
Avg Revenue: £689

**Treatment Group**
1,947 customers
Avg Revenue: £3,731

**Revenue Difference**
+£3,042 per customer
in favour of treatment group

### 7.3 Interpretation

The 441.77% difference in average revenue per user between groups reflects the natural revenue concentration in high-value customer segments, not a directly measured causal effect of recommendations. Champions and Loyal customers spend significantly more by nature, as they are the retailer's best customers regardless of recommendations.

This framework establishes the business case: these high-value segments are the right target for personalised recommendation campaigns. In a production deployment, a true randomised controlled trial would be required to isolate the incremental revenue attributable specifically to recommendations. The framework here demonstrates the A/B test methodology and identifies where recommendation investment would have the highest return.

## 8. Recommender Systems

### 8.1 Rule-Based Recommender

#### How it works

The rule-based recommender uses co-purchase frequency analysis. For every pair of products bought together by the same customer, a co-purchase count is recorded. When recommending for a customer, the system finds all products they have bought, looks up the most frequently co-purchased products for each, aggregates scores for items not yet bought, and returns the top 5.

| Metric | Value |
|---|---|
| Total customer-level product pairs | 4,389,649 |
| Unique products in lookup | 3,877 |
| Customers with recommendations | 4,338 / 4,338 (100%) |
| Unique products recommended | 250 |
| Avg product revenue of recommendations | £62,891 |
| Avg unique buyers of recommended products | 670.6 |

#### Sample Output

Customer 12347, rule-based recommendations:

- PACK OF 72 RETROSPOT CAKE CASES (score: 4,491)
- WHITE HANGING HEART T-LIGHT HOLDER (score: 3,732)
- PARTY BUNTING (score: 3,523)
- SET OF 3 CAKE TINS PANTRY DESIGN (score: 3,372)

- JUMBO BAG RED RETROSPOT (score: 3,027)

## 8.2 Collaborative Filtering

### How it works

Collaborative filtering uses a user-item matrix where rows are customers, columns are products, and values are quantity purchased. Cosine similarity (via KNN with k=11) finds the 10 most similar customers. Recommendations are weighted by customer similarity and the similar customers' purchase quantities, returning products not yet bought by the target customer.

| Metric | Value |
|---|---|
| User-item matrix shape | 4,338 customers x 3,877 products |
| Algorithm | KNN with cosine similarity (k=11) |
| Customers with recommendations | 4,338 / 4,338 (100%) |
| Unique products recommended | 1,269 |
| Avg product revenue of recommendations | £12,968 |
| Avg unique buyers of recommended products | 248.5 |

### Sample Output

Customer 12347, collaborative filtering recommendations:

- DOUGHNUT LIP GLOSS (score: 77.46)
- 10 COLOUR SPACEBOY PEN (score: 38.08)
- GROW A FLYTRAP OR SUNFLOWER IN TIN (score: 37.54)
- PACK OF 12 SPACEBOY TISSUES (score: 33.20)
- LUNCH BAG RED RETROSPOT (score: 28.88)

## 8.3 Content-Based Recommender

### How it works

The content-based recommender uses TF-IDF (Term Frequency-Inverse Document Frequency) to vectorise product descriptions. Cosine similarity between product vectors identifies similar products based on their names. For each customer, similar products to ones they have already bought are aggregated and ranked.

| Metric | Value |
|---|---|
| TF-IDF vocabulary size | 1,971 terms |
| Similarity matrix shape | 3,877 x 3,877 |

| | |
|---|---|
| Customers with recommendations | 4,338 / 4,338 (100%) |
| Unique products recommended | 2036 |
| Avg product revenue of recommendations | £6,477 |

### Sample Output

Product: WHITE HANGING HEART T-LIGHT HOLDER, content-based similar products:

- PINK HANGING HEART T-LIGHT HOLDER (similarity: 0.8334)
- RED HANGING HEART T-LIGHT HOLDER (similarity: 0.8186)
- HANGING HEART ZINC T-LIGHT HOLDER (similarity: 0.768)
- CREAM HANGING HEART T-LIGHT HOLDER (similarity: 0.768)
- HEART T-LIGHT HOLDER (similarity: 0.7619)

## 8.4 Method Comparison

| Metric | Rule-Based | Collaborative Filtering | Content-Based |
|---|---|---|---|
| Unique products recommended | 250 | 1,269 | 2,036 |
| Customers covered | 4,338 | 4,338 | 4,338 |
| Avg product revenue | £62,891 | £12,968 | £6,477 |
| Avg unique buyers | 670.6 | 248.5 | 154.4 |
| Agreement with rule-based | N/A | 1.8% | N/A |
| Approach | Co-purchase frequency | Customer similarity (KNN) | Product name similarity (TF-IDF) |

### Key Insights

- Rule-based recommends only 250 unique products, concentrating on bestsellers and popular co-purchases, making it safe but less personalised
- Collaborative filtering recommends 5x more diverse products (1,269 unique), surfacing niche items based on similar customer behaviour
- The two methods agree only 1.8% of the time, confirming they capture fundamentally different signals
- Rule-based recommends higher-revenue products on average (£62,891 vs £12,968), pushing popular and proven items
- Collaborative filtering is more personalised but recommends less commercially proven products
- A hybrid approach combining both signals would be optimal in a production environment

# 9. AI Layer: LLM Integration

## 9.1 Overview

To elevate the project beyond traditional ML, two AI features were integrated using the Anthropic Claude API (claude-sonnet-4-20250514). These features transform the system from returning product lists to providing human-readable insights and conversational shopping assistance.

## 9.2 Feature 1: AI Recommendation Explanations

When the /recommend/all/ endpoint is called, the rule-based recommendations are passed to Claude along with the customer ID and method description. Claude generates a 2-3 sentence business-focused explanation of why those products were recommended and what they reveal about the customer's shopping preferences.

## 9.3 Feature 2: Conversational Recommender Chatbot

A /chat POST endpoint accepts a natural language message and optional customer ID. Claude reads the customer's purchase history and responds conversationally with personalised product recommendations. This simulates a real-time shopping assistant that understands context, budget constraints, and purchase history.

### Sample Chatbot Interaction

Input: 'I am looking for a gift under 20 pounds' (Customer 17850)

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(venv) PS C:\Users\neeha\Documents\Projects on Resume\ecommerce_recommendation\api> Invoke-RestMethod -Uri "http://127.0.0.1:5000/chat" -Method POST -ContentTy
pe "application/json" -Body '{"message": "I am looking for a gift under 20 pounds", "customer_id": 17850}'

customer_id message                              response
----------- -------                              --------
      17850 I am looking for a gift under 20 pounds Based on your lovely purchase history, I can see you have great taste for vintage and cozy items! Here ...


(venv) PS C:\Users\neeha\Documents\Projects on Resume\ecommerce_recommendation\api>
```

The chatbot correctly identifies the customer's affinity for heart motifs and home decor from their purchase history and makes relevant budget-conscious suggestions.

# 10. Flask API

## 10.1 Architecture

The recommender system is wrapped in a Flask REST API, making all three recommendation methods and the AI features accessible via HTTP endpoints. The API loads all models at startup and serves requests in real time.

## 10.2 Endpoints

| Method | Endpoint | Description |
|---|---|---|
| GET | / | API homepage with route documentation |
| GET | /recommend/rule/<customer_id> | Rule-based recommendations for a customer |
| GET | /recommend/collaborative/<customer_id> | Collaborative filtering recommendations |
| GET | /recommend/content/<customer_id> | Content-based recommendations |
| GET | /recommend/all/<customer_id> | All three methods + AI explanation |
| POST | /chat | Conversational recommender chatbot |

## 10.3 Sample API Response

127.0.0.1:5000/recommend/all/17850

Pretty-print


```
{
  "ai_explanation": "Based on rule-based co-purchase analysis, these products were recommended because Customer 17850's purchase history indicates a strong preference for party hosting and home entertaining items. The combination of tiered cake stands, decorative
bunting, ornamental accessories, and storage solutions suggests this customer frequently shops for special occasions, celebrations, or home decorating projects. This customer profile represents high value for cross-selling complementary party supplies and seasonal
home d\u00e9cor items.",
  "collaborative_filtering": [
    "RED HANGING HEART T-LIGHT HOLDER",
    "T-LIGHT HOLDER SWEETHEART HANGING",
    "WORLD WAR 2 GLIDERS ASSTD DESIGNS",
    "HOME BUILDING BLOCK WORD",
    "DECORATIVE WICKER HEART SMALL"
  ],
  "content_based": [
    "EDWARDIAN PARASOL PINK",
    "WOOD BLACK BOARD ANT WHITE FINISH",
    "3 DRAWER ANTIQUE WHITE WOOD CABINET",
    "EDWARDIAN PARASOL NATURAL",
    "VINTAGE BILLBOARD MUG "
  ],
  "customer_id": 17850,
  "rule_based": [
    "REGENCY CAKESTAND 3 TIER",
    "PARTY BUNTING",
    "ASSORTED COLOUR BIRD ORNAMENT",
    "JUMBO BAG RED RETROSPOT",
    "HEART OF WICKER SMALL"
  ]
}
```


## 10.4 Running the API

The API runs locally with a Python virtual environment. Dependencies are managed via requirements.txt. The Anthropic API key is stored in a .env file and never committed to version control.

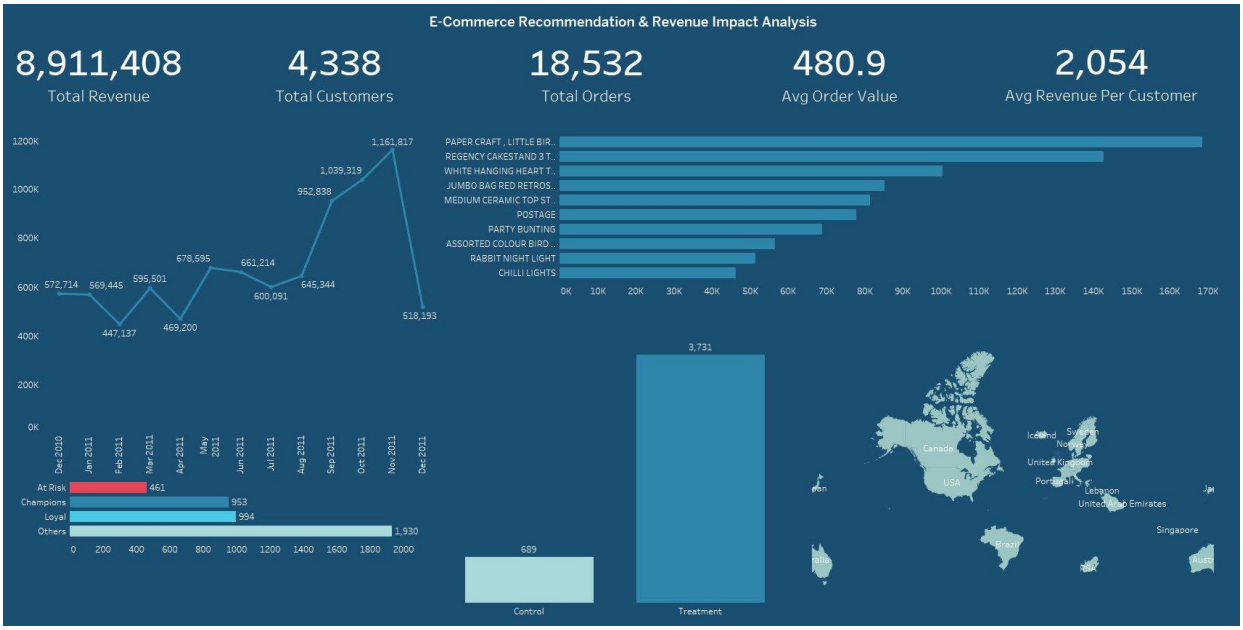
# 11. Tableau Dashboard

## 11.1 Overview

A Tableau dashboard was built to communicate the business insights from the analysis to non-technical stakeholders. The dashboard uses a dark professional theme (navy background with teal accents) and is organised into a single view covering all key metrics.

## 11.2 Dashboard Layout

| Section | Chart Type | Data Source |
|---|---|---|
| KPI Row (top) | 5 big number cards | kpi_summary.csv |
| Middle left | Line chart: Monthly Revenue trend | monthly_revenue.csv |
| Middle right | Horizontal bar: Top 10 Products | product_popularity.csv |
| Bottom left | Horizontal bar: Customer Segments | rfm_segments.csv |
| Bottom middle | Bar chart: A/B Test Results | ab_test_summary.csv |
| Bottom right | Map: Revenue by Country | country_revenue.csv |




# 12. SQL Pipeline

## 12.1 Overview

A complete SQLite pipeline was built to replicate all Python analysis steps in SQL. The pipeline covers data cleaning, KPI calculation, top products, monthly revenue, RFM scoring, customer segmentation, product co-purchase pairs, rule-based recommender, and A/B test framework.

## 12.2 Python and SQL: Division of Responsibility

Python handled all data cleaning and preprocessing, covering type conversions, filtering invalid rows, adding derived columns, and standardising date formats. This is the ETL (Extract, Transform, Load) layer. SQLite handled all analytics, including aggregations, window functions, joins, and business logic. This clean separation means the SQL queries are readable, maintainable, and free from data quality concerns.

Importing a Python-cleaned CSV into SQLite is consistent with how production data pipelines are structured: raw data is cleaned by a data engineering layer before being loaded into an analytical database for querying by analysts and BI tools.

## 12.3 SQL Pipeline Sections

- Section 1-2: Build clean_data table and KPI metrics
- Section 3-5: Top products, countries, monthly revenue
- Section 6: RFM raw metrics using MAX(JULIANDAY(InvoiceDate_ISO))
- Section 7: RFM scoring using NTILE(5) window functions
- Section 8: Customer segmentation with CASE WHEN logic

- Section 9-10: Product co-purchase pairs and rule-based recommender
- Section 11: A/B test framework with simulated uplift

# 13. Key Findings & Business Insights

## 13.1 Customer Insights

- Only 22% of customers (953) qualify as Champions, buying recently, frequently, and at high value. These are the highest priority for retention campaigns.
- 23% (994) are Loyal customers, consistent buyers who could be upgraded to Champions with targeted recommendations.
- 11% (461) are At Risk, previously good customers who have gone quiet. Reactivation campaigns with personalised recommendations are recommended.
- 44% (1,930) fall into Others, infrequent or low-value customers. Cost of acquisition likely exceeds lifetime value for this group.

## 13.2 Product Insights

- The top 10 products account for a disproportionate share of revenue, a classic long-tail distribution.
- Paper Craft Little Birdie is anomalous, with £168K revenue from a single buyer (80,995 units in one order). This is likely a wholesale/bulk order and may skew product popularity metrics.
- Heart-themed and decorative home items dominate the top products, suggesting a clear customer preference for gifting and home decor.

## 13.3 Recommender Insights

- Rule-based is better for driving revenue per recommendation (recommends proven bestsellers averaging £62,891 product revenue).
- Collaborative filtering is better for personalisation and discovery (5x more diverse, 1,269 vs 250 unique products).
- Only 1.8% overlap between methods confirms they are genuinely complementary, and a hybrid approach would capture both signals.
- Content-based filtering adds a third signal based purely on product name similarity, useful for new customers with limited purchase history.

## 13.4 Revenue Insights

- Clear seasonal pattern with Q4 2011 (Sep-Nov) accounting for 33% of annual revenue.
- UK dominates at 82% of revenue, meaning international expansion represents a significant growth opportunity.
- Average order value of £480.87 is high, suggesting bulk/wholesale buyers in the customer base alongside retail customers.

## 14. Technical Stack

| Component | Technology | Version/Notes |
|---|---|---|
| Language | Python | 3.10+ |
| Data manipulation | Pandas | 2.x |
| Numerical computing | NumPy | 1.x |
| Machine learning | Scikit-learn | KNN, TF-IDF, cosine similarity |
| Sparse matrices | SciPy | csr_matrix |
| LLM API | Anthropic Claude | claude-sonnet-4-20250514 |
| Web framework | Flask | 3.x |
| Environment management | python-dotenv | API key management |
| Database | SQLite | Via DB Browser for SQLite |
| BI Dashboard | Tableau Desktop Public | 2024.x |
| Development | VS Code | With Python extension |
| Notebook | Google Colab | Data exploration and export |
| Version control | Git / GitHub | See project repository |

## 15. Project Structure

```
ecommerce-recommendation-project/ ├── data/ │   └── data_clean.csv ├── notebooks/ │
└── ecommerce_analysis.py ├── sql/ │   └── pipeline.sql ├── api/ │   ├── app.py │   ├──
requirements.txt │   ├── .env  (not committed to Git) │   └── .gitignore ├── outputs/ │
├── tableau/ │   │   ├── kpi_summary.csv │   │   ├── monthly_revenue.csv │   │   ├──
country_revenue.csv │   │   ├── product_popularity.csv │   │   ├── rfm_segments.csv │   │
└── ab_test_summary.csv │   ├── recommendations/ │   │   ├──
rule_based_recommendations.csv │   │   ├── collab_recommendations.csv │   │   └──
content_based_recommendations.csv │   └── comparison/ │       ├──
recommender_comparison.csv │       └── method_comparison_summary.csv └── tableau/
└── ecommerce_dashboard.twbx
```

*End of Report*

[SATYA SAHITHI VAKKALANKA] | [NEEHAR SATTI] | April 2026